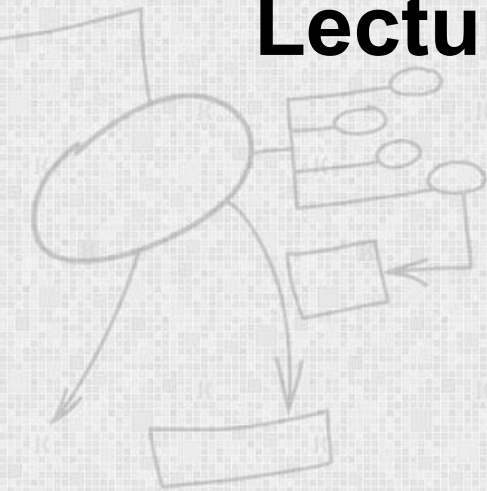


Database Engineering

Lecture 9: Distributed Databases

By

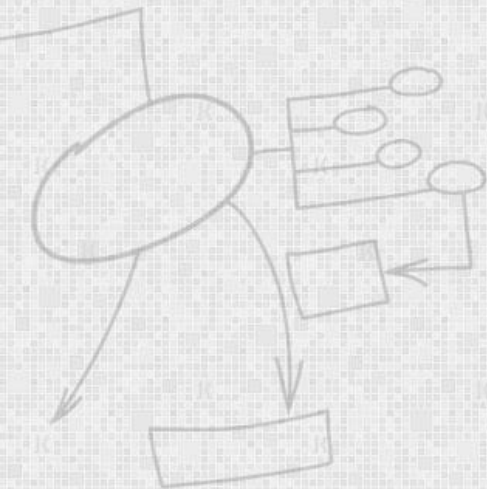
Dr Wasi Haider Butt



Distributed Database

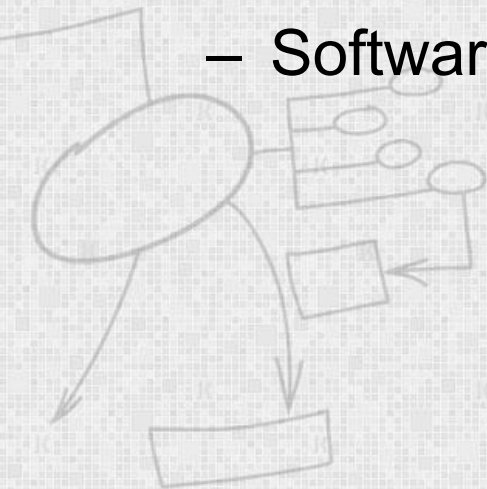
Physically located at different sites, Cooperating to share data. May act as single database

- Top down approach
- Bottom Up approach

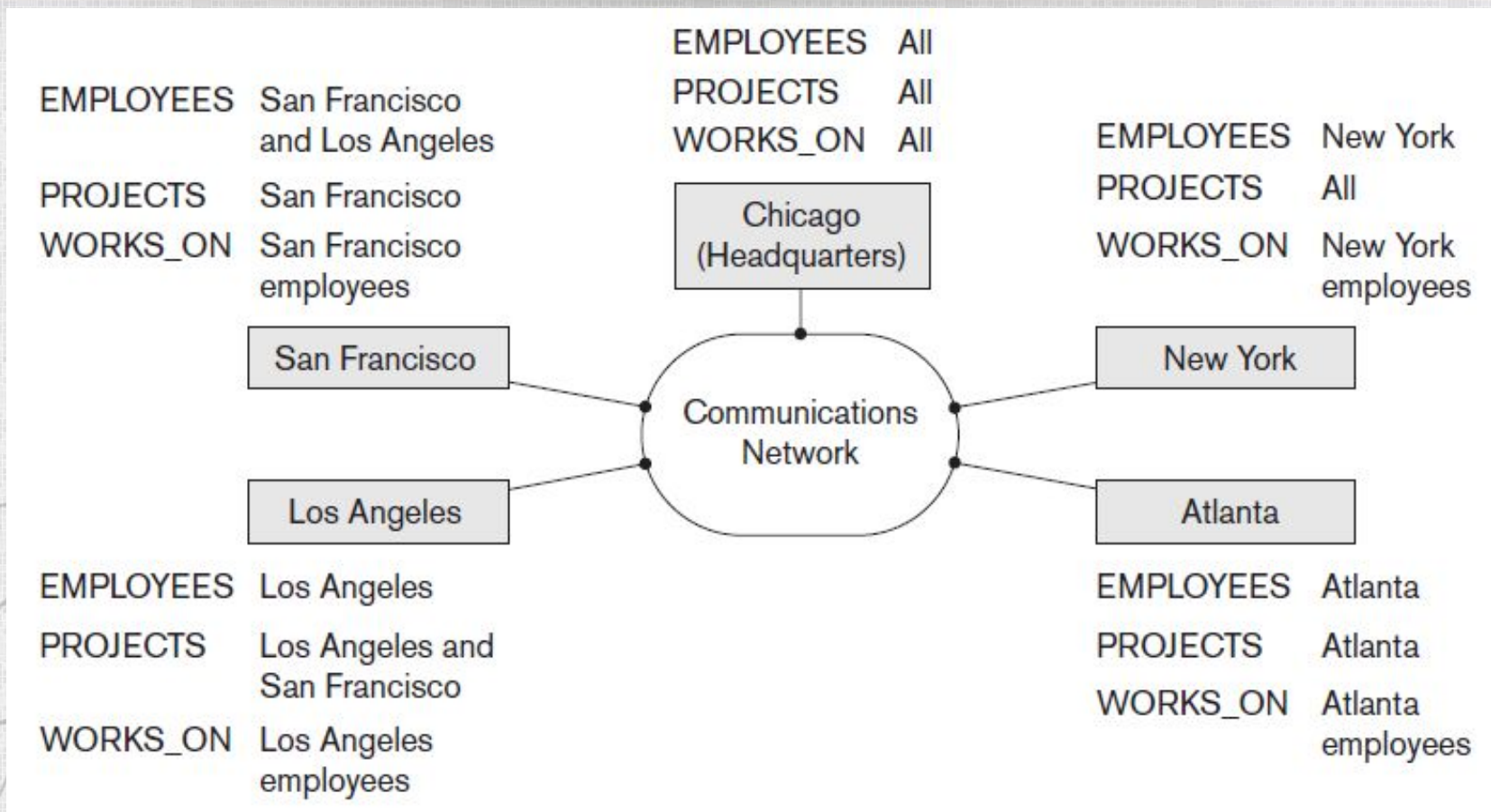


Distributed Database Concepts

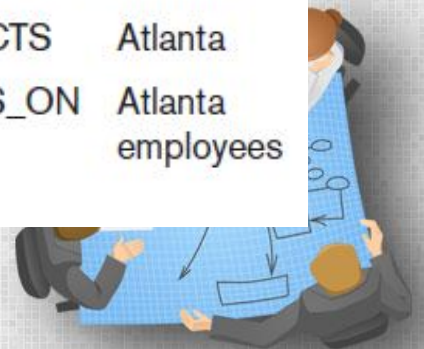
- What constitutes a distributed database?
 - Connection of database nodes over computer network
 - Logical interrelation of the connected databases
- Distributed database management system (DDBMS)
 - Software system that manages a distributed database



Distributed Databases



Data distribution and replication among distributed databases



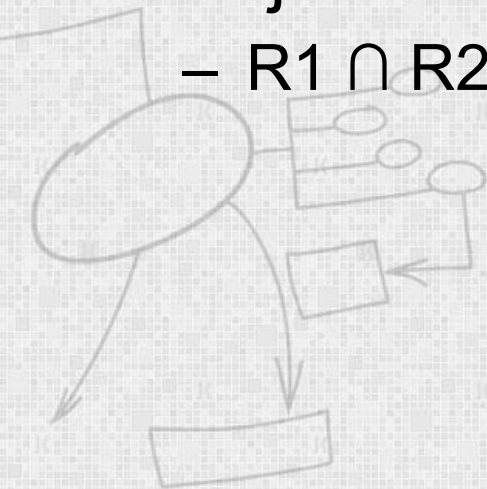
Data Fragmentation, Replication, and Allocation Techniques for Distributed Database Design

- Fragments
 - Units of the database
- Horizontal fragmentation (sharding)
 - Horizontal fragment or shard of a relation is a subset of the tuples in that relation
 - Can be specified by condition on one or more attributes or by some other method
 - Groups rows to create subsets of tuples
 - Each subset has a certain logical meaning



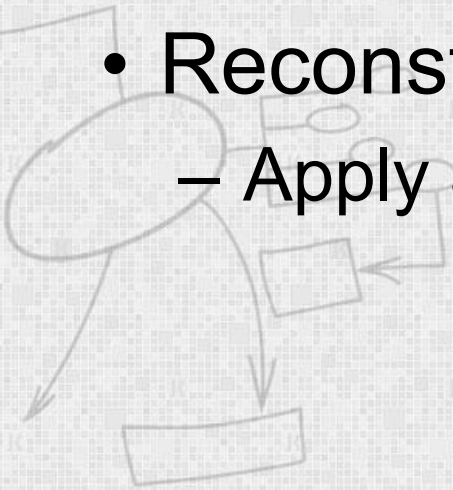
Horizontal Fragmentation – Correctness

- Completeness
 - No tuple is missing
- Reconstruction
 - Original relation can be reconstructed using some relational operation (Θ)
- Disjointness
 - $R_1 \cap R_2 \cap \dots \cap R_n = \text{null}$



Data Fragmentation (cont'd.)

- Vertical fragmentation
 - Divides a relation vertically by columns
 - Keeps only certain attributes of the relation
- Completeness
 - No Attribute is Missing
- Reconstruction
 - Apply JOIN operation to reconstruct relation



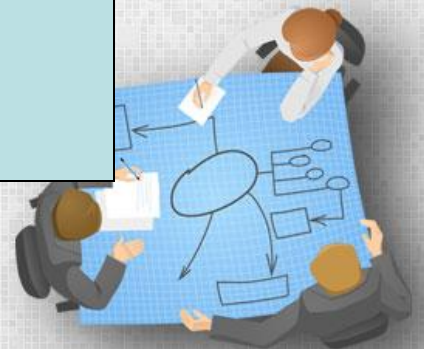
Horizontal Fragmentation of *account* Relation

<i>branch_name</i>	<i>account_numbe</i>	<i>balance</i>
<i>r</i>		
Hillside	A-305	500
Hillside	A-226	336
Hillside	A-155	62

$$account_1 = \sigma_{branch_name="Hillside"}(account)$$

<i>branch_name</i>	<i>account_numbe</i>	<i>balance</i>
<i>r</i>		
Valleyview	A-177	205
Valleyview	A-402	10000
Valleyview	A-408	1123
Valleyview	A-639	750

$$account_2 = \sigma_{branch_name="Valleyview"}(account)$$



Vertical Fragmentation of *employee_info* Relation

<i>branch_name</i>	<i>customer_name</i>	<i>tuple_id</i>
Hillside	Lowman	1
Hillside	Camp	2
Valleyview	Camp	3
Valleyview	Kahn	4
Hillside	Kahn	5
Valleyview	Kahn	6
Valleyview	Green	7

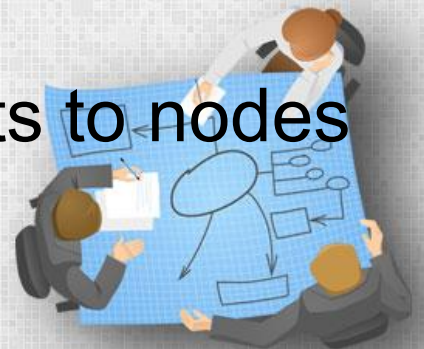
$deposit_1 = \Pi_{branch_name, customer_name, tuple_id}(employee_info)$

<i>account_numbe</i>	<i>balance</i>	<i>tuple_id</i>
A-305	500	1
A-226	336	2
A-177	205	3
A-402	10000	4
A-155	62	5
A-408	1123	6
A-639	750	7

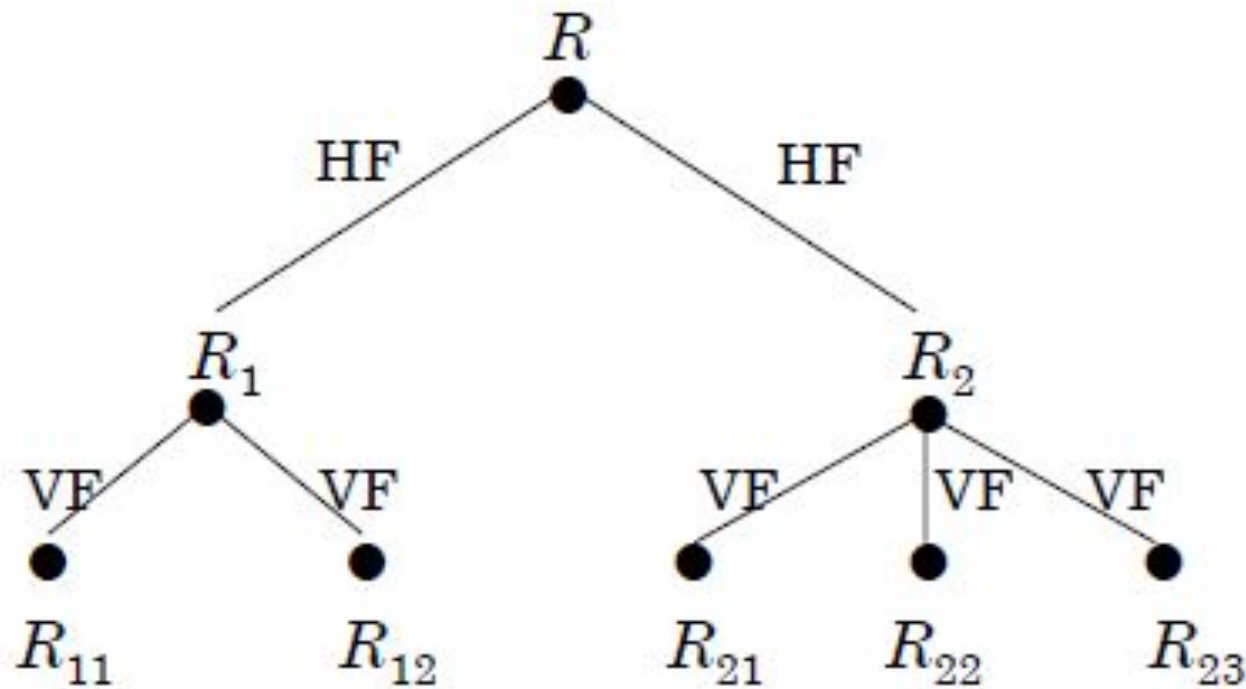
$deposit_2 = \Pi_{account_number, balance, tuple_id}(employee_info)$

Data Fragmentation (cont'd.)

- Mixed (hybrid) fragmentation
 - Combination of horizontal and vertical fragmentations
- Fragmentation schema
 - Database design in terms of fragments
 - $R = R_1 \cup R_2$
- Allocation schema
 - Describes the allocation of fragments to nodes of the DDBS
 - R_1 is on S_1 and R_2 is on S_2

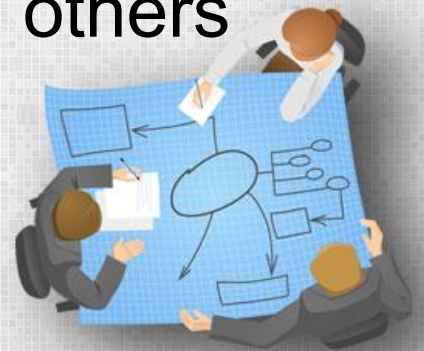


Hybrid Fragmentation



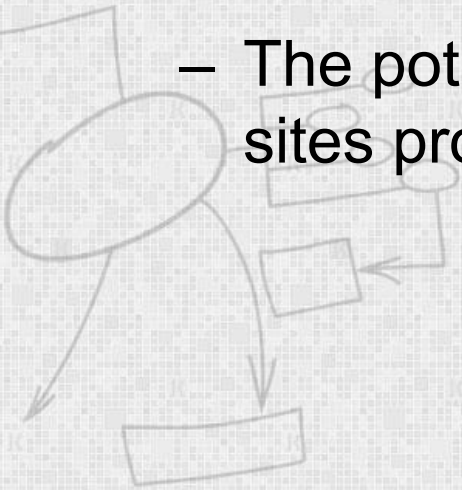
Data Replication and Allocation

- Fully replicated distributed database
 - Replication of whole database at every site in distributed system
 - Improves availability remarkably
 - Update operations can be slow
- Partial replication
 - Some fragments are replicated, and others are not
 - Defined by replication schema



Distributed Query Processing

- For centralized systems, the primary criterion for measuring the cost of a particular strategy is the local resources utilized.
- In a distributed system, other issues must be taken into account:
 - The cost of a data transmission over the network.
 - The potential gain in performance from having several sites process parts of the query in parallel.

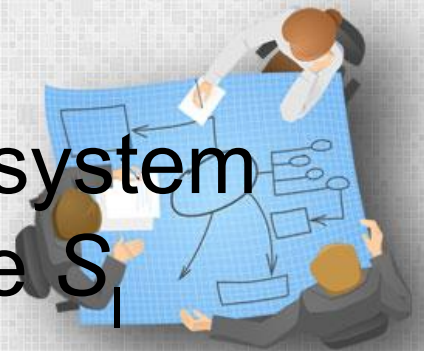


Simple Join Processing

- Consider the following expression in which the three relations are neither replicated nor fragmented

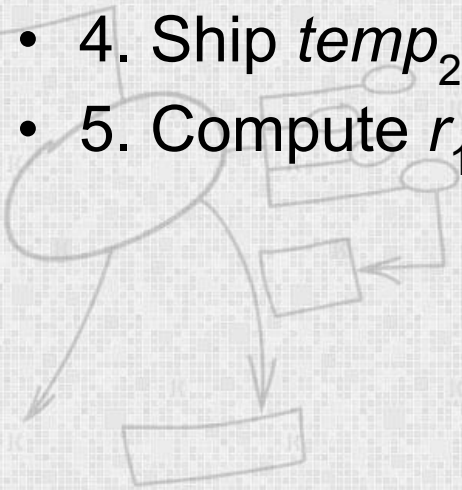
account ⋈ *depositor* ⋈ *branch*

- *account* is stored at site S_1
- *depositor* at S_2
- *branch* at S_3
- For a query issued at site S_1 , the system needs to produce the result at site S_1



Semijoin Strategy

- Let r_1 be a relation with schema R_1 stores at site S_1
Let r_2 be a relation with schema R_2 stores at site S_2
 - Evaluate the expression $r_1 \bowtie r_2$ and obtain the result at S_1 .
1. Compute $temp_1 \leftarrow \Pi_{R_1 \cap R_2}(r_1)$ at S_1 .
 - 2. Ship $temp_1$ from S_1 to S_2 .
 - 3. Compute $temp_2 \leftarrow r_2 \bowtie temp_1$ at S_2
 - 4. Ship $temp_2$ from S_2 to S_1 .
 - 5. Compute $r_1 \bowtie temp_2$ at S_1 . This is the same as $r_1 \bowtie r_2$.



Formal Definition

- The **semijoin** of r_1 with r_2 , is denoted by:

$$r_1 \bowtie r_2$$

- it is defined by:

$$\Pi_{R1} (r_1 \bowtie r_2)$$

- Thus, $r_1 \bowtie r_2$ selects those tuples of r_1 that contributed to $r_1 \bowtie r_2$.
- For joins of several relations, the above strategy can be extended to a series of semijoin steps.

